

BCAC 0026: ANALYSIS OF ALGORITHM AND DATA STRUCTURES

Faculty Name

Dr. Vinod Jain, Assistant Professor, CEA

Mob : 9813078438, Email – vinod.jain@gla.ac.in

Seating : 329-B, AB-X, Third Floor

Course Structure

Credits: 04

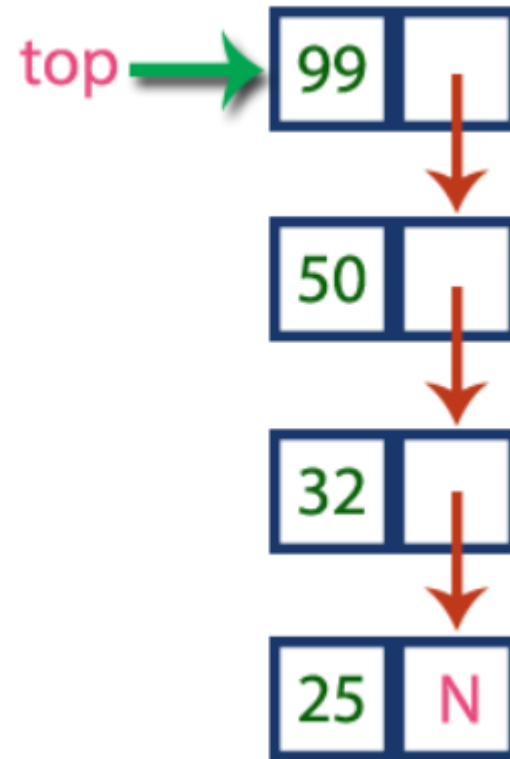
L-T-P: 4-0-0

Module No.	Content	Teaching Hours
I	Introduction: Basic Design and Analysis techniques of Algorithms, time and space complexity, Correctness of Algorithm. Algorithm Design Techniques: Iterative techniques, Divide and Conquer, Dynamic Programming, Greedy Algorithms. Sorting Techniques: Elementary sorting techniques-Bubble Sort, Insertion Sort, Merge Sort, Advanced Sorting techniques-Heap Sort, Quick Sort, Sorting in Linear Time-Bucket Sort, Radix Sort and Count Sort. Searching Techniques and Complexity Analysis: Linear and Binary search, Medians & Order Statistics. Arrays: Single and Multi-dimensional Arrays, Sparse Matrices.	30
II	Stacks and Queues: Implementing stack using array and linked list, Prefix, Infix and Postfix expressions, Utility and conversion of these expressions from one to another; Array and Linked representation of Queue, De-queue, Priority Queues. Linked Lists: Singly, Doubly and Circular Lists, representation of Stack and Queue as Linked Lists. Recursion: Developing Recursive Definition of Simple Problems and their implementation; Advantages and Limitations of Recursion. Trees: Introduction to Tree as a data structure; Binary Trees, Binary Search Tree, (Creation, and Traversals of Binary Search Trees).	30

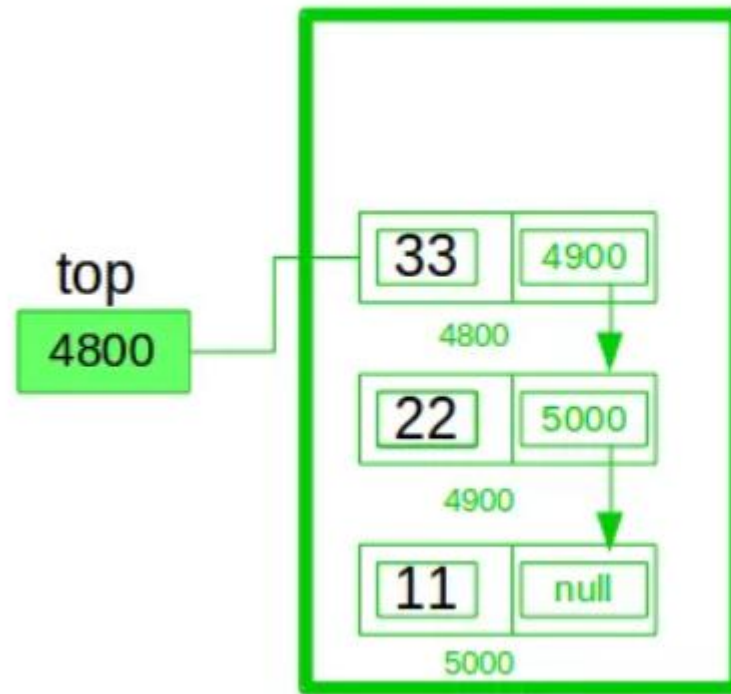
Linked Representation of Stack and Queue

- Stack using Linked List
- Queue using Linked List

Stack using Linked List



Stack using Linked List



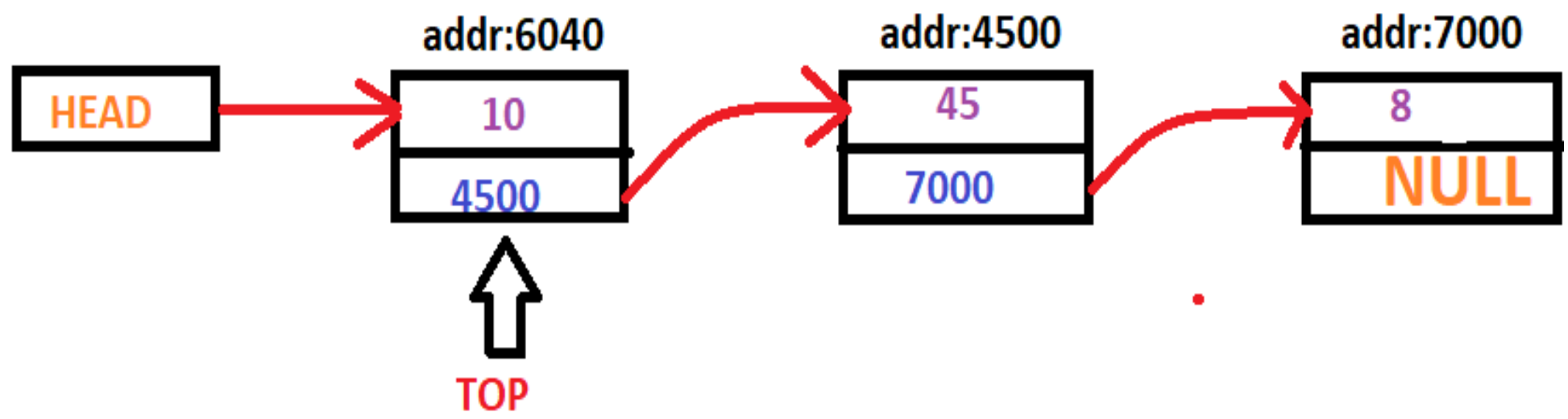
Initial Stack Having Three element
And top have address 4800

REPRESENTING STACK
USING LIST

STACK:



LIST:



Stack using Linked List

- In data structures,
- a **stack** is a linear data structure that follows the **Last In First Out (LIFO)** principle
- — the last element added to the stack is the first one to be removed.

Stack using Linked List

- The **linked representation of a stack** uses a **linked list** instead of an array. Each element (node) in the stack contains two parts:
 - **Data** – The actual value.
 - **Pointer (next)** – A reference to the next node in the stack.
-
- The top of the stack is represented by a pointer called top, which points to the most recently added node.

Stack using Linked List

- **Structure of Node**
- class Node {
 - int data;
 - Node* next;
- };

Stack using Linked List

- **How It Works**
 - **1. Push Operation (Insert an element)**
 1. Create a new node.
 2. Assign data to it.
 3. Set its next pointer to the current top.
 4. Update top to point to this new node.

Stack using Linked List

- **Pop Operation (Remove an element)**

1. Check if top is NULL (stack underflow).
2. Store top in a temporary variable.
3. Move top to top->next.
4. Free the memory of the removed node.

Stack using Linked List

- **Advantages**
- No fixed size: Stack grows dynamically as memory is allocated at runtime.
- Efficient memory usage: Only the required amount of memory is used.
- No overflow unless memory is exhausted.

Stack using Linked List

- **Disadvantages**
- More memory per element due to storing pointer along with data.
- Slightly slower than array representation due to dynamic memory allocation and pointer handling.

Stack using Linked List

- **Applications**
- Function call stack in recursion.
- Undo features in editors.
- Expression evaluation (postfix, prefix).
- Syntax parsing in compilers.

Linked Stack Implementation in Java

```
// Node class representing each element in the
stack
class Node {
    int data;
    Node next;

    // Constructor
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

// Stack class using linked list

```
class LinkedStack {  
    private Node top;
```

// Constructor

```
public LinkedStack() {  
    top = null;  
}
```

// Push operation

```
public void push(int value) {  
}
```

// Pop operation

```
public int pop() {  
}
```

// Peek operation

```
public int peek() {  
}
```

// Check if stack is empty

```
public boolean isEmpty() {  
    return top == null;  
}
```

// Display stack elements

```
public void display() {  
}  
}
```



```
// Stack class using linked list
class LinkedStack {
    private Node top;

    // Constructor
    public LinkedStack() {
        top = null;
    }
}
```

```
// Push operation
    public void push(int value) {
        Node newNode = new Node(value);
        newNode.next = top;
        top = newNode;
        System.out.println(value + "pushed to stack.");
    }
```

```
// Pop operation
```

```
    public int pop() {
```

```
        if (isEmpty()) {
```

```
            System.out.println("Stack Underflow!");
```

```
            return -1;
```

```
        }
```

```
        int popped = top.data;
```

```
        top = top.next;
```

```
        System.out.println(popped + " popped from stack.");
```

```
        return popped;
```

```
    }
```

```
// Peek operation
public int peek() {
    if (isEmpty()) {
        System.out.println("Stack is empty.");
        return -1;
    }
    return top.data;
}
```

```
// Check if stack is empty
public boolean isEmpty() {
    return top == null;
}
```

```
// Display stack elements
public void display() {
    if (isEmpty()) {
        System.out.println("Stack is empty.");
        return;
    }
    System.out.print("Stack elements: ");
    Node temp = top;
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
}
```

```
// Main class to test the stack
public class LinkedStackDemo {
    public static void main(String[] args) {
        LinkedStack stack = new LinkedStack();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        stack.display();
        // Output: 30 20 10
        stack.pop();
        stack.display();
        // Output: 20 10
        System.out.println("Is stack empty? " +
            stack.isEmpty()); // Output: false
    }
}
```

Linked Representation of Queue OR Queue using Linked List

- A

Stack using Linked List : Program

```

• Linked Stack Implementation in Java

• // Node class representing each element in the stack

• class Node {

•     int data;

•     Node next;

•

•     // Constructor

•     public Node(int data) {

•         this.data = data;

•         this.next = null;

•     }

• }

•

• // Stack class using linked list

• class LinkedStack {

•     private Node top;

•

•     // Constructor

•     public LinkedStack() {

•         top = null;

•     }

•

•     // Push operation

•     public void push(int value) {

•         Node newNode = new Node(value);

•         newNode.next = top;

•         top = newNode;

•         System.out.println(value + " pushed to stack.");

•     }

•

•     // Pop operation

•     public int pop() {

•         if (isEmpty()) {

•             System.out.println("Stack Underflow!");

•             return -1;

•         }

•         int popped = top.data;

•         top = top.next;

•         System.out.println(popped + " popped from stack.");

•         return popped;

•     }

•

•     // Peek operation

•     public int peek() {

•         if (isEmpty()) {

•             System.out.println("Stack is empty.");

•             return -1;

•         }

•     }

•

•     return top.data;

• }

• // Check if stack is empty

• public boolean isEmpty() {

•     return top == null;

• }

• // Display stack elements

• public void display() {

•     if (isEmpty()) {

•         System.out.println("Stack is empty.");

•         return;

•     }

•     System.out.print("Stack elements: ");

•     while (temp != null) {

•         System.out.print(temp.data + " ");

•         temp = temp.next;

•     }

•     System.out.println();

• }

• // Main class to test the stack

• public class LinkedStackDemo {

•     public static void main(String[] args) {

•         LinkedStack stack = new LinkedStack();

•

•         stack.push(10);

•         stack.push(20);

•         stack.push(30);

•         stack.display();    // Output: 30 20 10

•

•         System.out.println("Top element is: " + stack.peek()); // Output: 30

•

•         stack.pop();

•         stack.display();    // Output: 20 10

•

•         System.out.println("Is stack empty? " + stack.isEmpty()); // Output: false

•

•         stack.pop();

•         stack.pop();

•         stack.pop();    // Trying to pop from empty stack

•     }

• }

```


Linked List Representation of a Queue



Queue using Linked List

- A **queue** is a linear data structure that follows the **First In First Out (FIFO)** principle — the first element added to the queue is the first one to be removed.

Queue using Linked List

- **Linked Representation**
- In a **linked list representation**, each node of the queue contains:
 - **Data**: The actual value.
 - **Next pointer**: A reference to the next node.
- Two pointers are maintained:
 - **Front**: Points to the front (first) node.
 - **Rear**: Points to the rear (last) node.

Queue using Linked List

- Structure of Node:
- class Node {
- int data;
- Node next;
- }

Queue using Linked List

- **Operations**
 - 1. Enqueue (Insert an element at the rear)
 - 2. Dequeue (Remove an element from the front)
 - 3. Peek (Get front element)
 - 4. IsEmpty (Check if queue is empty)

Enqueue

- 1. Enqueue (Insert an element at the rear)
 - • Create a new node.
 - • If the queue is empty, both front and rear point to the new node.
 - • Else, add the new node at the end and move the rear pointer.

Deque

- 2. Dequeue (Remove an element from the front)
 - • If the queue is empty, report underflow.
 - • Remove the node pointed to by front.
 - • Move front to the next node.
 - • If the queue becomes empty, set rear to null.

Queue using Linked List

- 3. Peek (Get front element)
 - Return the data at the front node.
- 4. IsEmpty (Check if queue is empty)
 - Return true if front is null.

Queue using Linked List

- Program

Complete Java Code for Linked Queue

// Node class

```
class Node {
```

```
    int data;
```

```
    Node next;
```

```
    public Node(int data) {
```

```
        this.data = data;
```

```
        this.next = null;
```

```
    }
```

```
}
```

```
// Queue class using linked list
class LinkedList {
    private Node front, rear;

    // Constructor
    public LinkedList() {
        front = rear = null;
    }
}
```

```
// Enqueue operation
public void enqueue(int value) {
    Node newNode = new Node(value);
    if (rear == null) {
        front = rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
    System.out.println(value + " enqueued to queue.");
}
```

```
// Dequeue operation
public int dequeue() {
    if (isEmpty()) {
        System.out.println("Queue Underflow!");
        return -1;
    }
    int result = front.data;
    front = front.next;
    if (front == null) rear = null;
    System.out.println(result + " dequeued from queue.");
    return result;
}
```

```
// Peek front element
public int peek() {
    if (isEmpty()) {
        System.out.println("Queue is empty.");
        return -1;
    }
    return front.data;
}

// Check if queue is empty
public boolean isEmpty() {
    return front == null;
}
```

```
// Display queue elements
public void display() {
    if (isEmpty()) {
        System.out.println("Queue is empty.");
        return;
    }
    System.out.print("Queue elements: ");
    Node temp = front;
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
```

```
// Main class to test the queue
public class LinkedQueueDemo {
    public static void main(String[] args) {
        LinkedQueue queue = new LinkedQueue();

        queue.enqueue(100);
        queue.enqueue(200);
        queue.enqueue(300);
        queue.display();    // Output: 100 200 300

        System.out.println("Front element is: " + queue.peek()); // Output: 100

        queue.dequeue();
        queue.display();    // Output: 200 300

        queue.dequeue();
        queue.dequeue();
        queue.dequeue();    // Queue underflow
    }
}
```


Queue using Linked List : Program

```

• Complete Java Code for Linked Queue
• // Node class
• class Node {
•     int data;
•     Node next;
•
•     public Node(int data) {
•         this.data = data;
•         this.next = null;
•     }
• }
•
• // Queue class using linked list
• class LinkedQueue {
•     private Node front, rear;
•
•     // Constructor
•     public LinkedQueue() {
•         front = rear = null;
•     }
•
•     // Enqueue operation
•     public void enqueue(int value) {
•         Node newNode = new Node(value);
•
•         if (rear == null) {
•             front = rear = newNode;
•         } else {
•             rear.next = newNode;
•             rear = newNode;
•         }
•         System.out.println(value + " enqueued to queue.");
•     }
•
•     // Dequeue operation
•     public int dequeue() {
•         if (isEmpty()) {
•             System.out.println("Queue Underflow!");
•             return -1;
•         }
•         int result = front.data;
•         front = front.next;
•         if (front == null) rear = null;
•         System.out.println(result + " dequeued from queue.");
•         return result;
•     }
•
•     // Peek front element
•     public int peek() {
•         if (isEmpty()) {
•             System.out.println("Queue is empty.");
•             return -1;
•         }
•         return front.data;
•     }
•
•     // Check if queue is empty
•     public boolean isEmpty() {
•         return front == null;
•     }
•
•     // Display queue elements
•     public void display() {
•         if (isEmpty()) {
•             System.out.println("Queue elements: ");
•             return;
•         }
•         System.out.print("Queue elements: ");
•         Node temp = front;
•         while (temp != null) {
•             System.out.print(temp.data + " ");
•             temp = temp.next;
•         }
•         System.out.println();
•     }
• }
•
• // Main class to test the queue
• public class LinkedQueueDemo {
•     public static void main(String[] args) {
•         LinkedQueue queue = new LinkedQueue();
•         queue.enqueue(100);
•         queue.enqueue(200);
•         queue.enqueue(300);
•         queue.display(); // Output: 100 200 300
•         queue.dequeue();
•         queue.display(); // Output: 200 300
•         queue.dequeue();
•         queue.dequeue(); // Queue underflow
•         System.out.println("Front element is: " + queue.peek()); // Output: 100
•     }
• }

```

Output:

vbnet

CopyEdit

100 enqueued to queue.

200 enqueued to queue.

300 enqueued to queue.

Queue elements: 100 200 300

Front element is: 100

100 dequeued from queue.

Queue elements: 200 300

200 dequeued from queue.

300 dequeued from queue.

Queue Underflow!

Queue using Linked List

- A